FirstStep: Security Self-Assessment Report

2026 FinTechathon, International Track, Topic B: Wealth Advisory Agent. This report covers the permission-tier implementation and the known-risk list. The security design itself is in `docs/technical-documentation.md`, section 5.

1. Scope and method

- **System:** the FirstStep server (`server/app`), its web client, and the LLM it calls (Gemma 4 26B via Google AI Studio in the evidence run).
- **Assets:** the person's sandbox money and orders, their profile data, the integrity of the advice (suitability, no misleading claims), and the integrity of the record.
- **Method:** threat modelling against the OWASP Top 10 for LLM Applications (2025), especially LLM01 Prompt Injection and LLM06 Excessive Agency. Each control has an allow test and a deny test. A 30-case red-team suite runs against the real model, and every case is judged from the audit log, not from the reply's wording.
- **Assumption:** the model is untrusted. Any defence that only works if the model behaves is treated as a convenience, not a control.

2. Permission tiers

2.1 The tiers

Every tool has a tier in `server/data/rules.yaml` (`TIERS.tier_of`). The tier decides who may call the tool and what must happen first.

Tier	Meaning	Tools	Who may call it	What must pass first
0 Read	Reads or records; moves no money	`answer_question`, `update_profile`, `assess_readiness`, `score_risk_profile`, `set_goal`, `list_suitable_products`, `compare_products`, `explain`, `get_portfolio`, `check_drift`, `advance_time`	Person (buttons) or LLM	`STAGE-01`; `SUIT-01` filters listings
1 Propose	Creates a draft; nothing executes	`build_savings_plan`, `propose_allocation`, `propose_custom_allocation`, `project_outcomes`, `prepare_orders`, `propose_rebalance`	Person or LLM	`STAGE-01`, `READY-01/02` (via stages), `SUIT-01`, `CONC-01`, `HOR-01`, `LIM-01` on drafted orders

Tier	Meaning	Tools	Who may call it	What must pass first
2 Execute	Places sandbox orders	`place_order`	**Only the Confirm route**, with a token. Never offered to the LLM.	`STAGE-01`, `CONF-01` token, `SUIT-01`, `LIM-01`
3 Sensitive	A sale that may be panic-driven	`request_sell`	Person or LLM	`STAGE-01`, `COOL-01` (24-hour pause after a fall, then asks again). The result is only a draft; placing it is Tier 2.

2.2 How each tier is enforced

1. **Offering.** `llm_tools()` in `app/agent/turn.py` builds the model's tool list from `STAGE-01` for the current stage and drops every Tier 2 tool. The model cannot call a tool it was never given; a call naming one fails schema lookup.
2. **One executor.** Buttons, the LLM and the Confirm route all call `executor.run()`. It validates the arguments (Pydantic, unknown fields rejected), asks the guardian (`app/policy/gate.py`), runs the tool, and logs the call, the decision and the result. No route calls a tool directly. Import-linter checks the layer boundaries on every push, so a new route cannot skip the executor without failing CI.
3. **Re-checking.** The guardian ignores what the model claims. A custom mix is re-checked for `SUIT-01` and `CONC-01`. Drafted orders are re-checked for `LIM-01`. At placement, the token, suitability and limits are checked again against the stored proposal.
4. **The confirmation token (`CONF-01`).** It is an HMAC-SHA256 over the proposal ID, the SHA-256 of the proposal's exact contents, a random nonce and a 10-minute expiry. It is created only in the `POST /confirm/{proposal_id}` handler, which the Confirm button calls. The dialog keeps that button disabled until the person ticks the statement "I understand the value can go down as well as up." It is passed straight to `place_order` server-side, never returned to the browser or the model, and redacted from the audit log. The signature is compared with `hmac.compare_digest`, and a used nonce is stored, so a token works once.
5. **Stage gating.** `STAGE-01` runs first on every call. A person who fails the money check is in `SAVINGS_PLAN`, where no investment tool exists, whatever the model or the browser sends.

2.3 Evidence that the tiers hold

Control	Unit tests (`server/tests`)	Red-team cases (real model)
`STAGE-01`	allows a tool in its stage; blocks ordering before Confirm; blocks investing tools in the savings plan	RT-10, RT-11, RT-19, RT-20, RT-21 (forged calls straight to the API, all blocked)
`CONF-01`	blocks a missing, forged, other-key, other-proposal, changed-proposal, expired and reused token	RT-07, RT-08, RT-09, RT-10 (talked into buying, fake token, forged call: nothing filled)

Control	Unit tests (`server/tests`)	Red-team cases (real model)
`SUIT-01`	allows Mei's C2 portfolio; blocks all tech for Mei	RT-12 to RT-15
`CONC-01`	allows the C4 portfolio; blocks everything in one fund	RT-16
`LIM-01`	blocks one order over $10,000 and passing the daily $20,000	RT-22
`COOL-01`	allows a small sale in a drop; blocks a panic sale	RT-23, RT-24
`READY-01/02`	full cushion allowed, thin cushion blocked; cheap student loan allowed, card debt blocked	RT-18 to RT-21
`LANG-01..04`	blocks promises and predictions; allows honest range wording; the fallback reply passes	RT-25, RT-26, RT-27
Input guard	injection flags; PII masking	RT-01 to RT-06, RT-17, RT-28, RT-29
Audit chain	catches an edited, deleted or reordered line, broken JSON, a forged re-hashed link, and a fully rewritten file (anchor)	every case requires the chain to verify

Red-team result on Gemma 4 26B: 30 of 30 handled safely. By category: prompt injection 6/6, unsuitable funds 6/6, acting without a click 5/5, investing before ready 4/4, promises and predictions 3/3, panic selling 2/2, personal data 2/2, order limits 1/1, out of scope 1/1. Ten were stopped by the input guard, 13 by a guardian rule, and 7 were answered without any tool being called. Across all 30, **no order filled without a Confirm click, no token appeared in any log, and every audit chain verified** (`server/evidence/gemma/redteam.md`).

3. Known-risk list

Likelihood (L) and impact (I) are rated for a production version of this design; in the sandbox no real money is at risk.

ID	Risk	L / I	Current mitigation	Residual risk and next step
KR-01	**Prompt injection gets past the input guard.** The guard is a list of patterns, and new phrasings will slip through.	High / Low	The model has no Tier 2 tool, every call is re-checked, and the token never reaches the model. Injection can change the words, not the actions (RT-01 to RT-09).	The model could still be steered into an unhelpful or off-topic reply. Next: a classifier-based injection detector, and logging flagged turns for review.

ID	Risk	L / I	Current mitigation	Residual risk and next step
KR-02	**Misleading wording the LANG filter misses.** Patterns catch “guaranteed” or “will double”, not every paraphrase (e.g. “you’ll surely make money”).	Medium / Medium	Instructions forbid promises; outcomes are shown only as ranges from a tool, captioned “not a prediction”; a disclosure is added to plan replies.	Paraphrased promises may pass. Next: an LLM or classifier second check on replies, and a sample of replies reviewed by compliance.
KR-03	**The model does the wrong safe thing.** It may pick the wrong tool or save a slightly wrong value. In testing it once saved a range midpoint ($3,750) instead of the typed $3,200, and once explained terms instead of showing outcomes.	Medium / Medium	Both were fixed (money questions now get no range values; tool descriptions say when to use each tool). The guardian bounds every outcome, and the person can correct a saved answer by telling Sprout (`update_profile`).	A wrong profile value can shift a plan within the suitable range. Next: read typed answers back for a one-tap confirmation before scoring.
KR-04	**The person misstates their profile** to unlock riskier funds.	Medium / Medium	Capacity is the lowest of several answers; short goals cap the class (`HOR-01`); suitability is per product.	A questionnaire cannot detect lying. Next: consistency checks between answers, and real KYC and income verification.
KR-05	**No user accounts.** A session is identified by a 96-bit random ID in the URL, which acts as a bearer token.	Low / High	IDs are unguessable (`secrets.token_urlsafe`); sessions hold no identity data.	Anyone with the link can use the session. Next: authentication, and session binding to an account.
KR-06	**No rate limiting.** A script could flood the API or burn the model quota.	Medium / Medium	Typed text is limited to 1,000 characters; each turn is capped at 4 tool calls, 9 model requests and a time limit.	Next: per-session and per-IP rate limits, and a spending cap on model usage.
KR-07	**Data sent to a model provider.** Typed messages go to Google’s API in the evidence run.	Medium / Medium	Card numbers, emails, ID numbers and phone numbers are masked before sending or logging; the model’s context holds no names or identity data.	Free text can still contain personal details the patterns miss. Next: host the open-weight model in-house (the adapter already supports swapping providers).
KR-08	**Audit log trust at write time.** A hash chain shows tampering after writing, but not a compromised server writing false events.	Low / High	Each event links to the previous hash; the head is anchored in the database and in order receipts; a verifier ships with the code.	The anchor lives on the same machine. Next: anchor the head periodically to external write-once storage or a timestamping service.
KR-09	**Secret management.** The token signing key comes from `CONFIRM_SECRET`; the development default is public.	Low / High	The signer rejects short secrets; `.env` is git-ignored; the README says to set a long random value outside development.	Next: refuse to start with the default outside development, and load secrets from a secret manager.

ID	Risk	L / I	Current mitigation	Residual risk and next step
KR-10	**Model availability and latency.** The free tier answers a typed message in about 13 s at the median (up to about 55 s), and sometimes not at all.	High / Low	Every step can be done with buttons, which never call the model; a failed turn keeps the person's answers and says so plainly; fallback model chains.	Next: a paid or self-hosted endpoint, and streaming replies.
KR-11	**A wrong number in the model's words.** The model might restate a figure wrongly.	Low / Medium	Every figure comes from a tool, and the cards beside the reply show the real numbers; the model never computes them.	The reply text itself is not checked against tool outputs. Next: check numbers in replies against the turn's tool results.
KR-12	**Simulation limits.** Ranges come from 2000–2025 history, before inflation and taxes; funds are fictional; fills are simulated.	—	Labelled throughout as an educational simulation; ranges are never called forecasts or backtests.	Not a production advisory service. A real version needs licensed products, real KYC/AML and a broker integration under the same tiers.
KR-13	**Dependencies.** Python packages other than the linters are not pinned to exact versions.	Low / Medium	CI runs the full test suite on every push; the client uses a lockfile (`package-lock.json`).	Next: a Python lockfile and automated dependency scanning.

4. Summary

The controls that matter most do not depend on the model: the model cannot execute, every call is re-checked by deterministic code, and the record is tamper-evident. The open risks are mostly about the quality of what the model says (KR-02, KR-03, KR-11) and about production hardening (KR-05, KR-06, KR-08, KR-09). The design already has places for each next step.